TinyIDS: end-to-end testbed design

Containers, the IDS pipeline, packet windows, compression / quantization, SHAP dashboard, laptop and Pi stages

Status: design (nothing built yet). Written 07 November 2026.

Contents

1 Why build the pipeline first **2**

2 The big picture **2**

3 The five containers **2**

4 Inside the IDS, step by step **4**

5 Packet windows: 10 and 100, like the dataset **4**

6 Compression and quantization (RQ2): a required step **6**

7 Swapping models: the contract **8**

8 Where SHAP and the dashboard run **8**

9 Stage A (laptop only) and stage B (Pi + NodeMCUs) **8**

10 Folder layout **10**

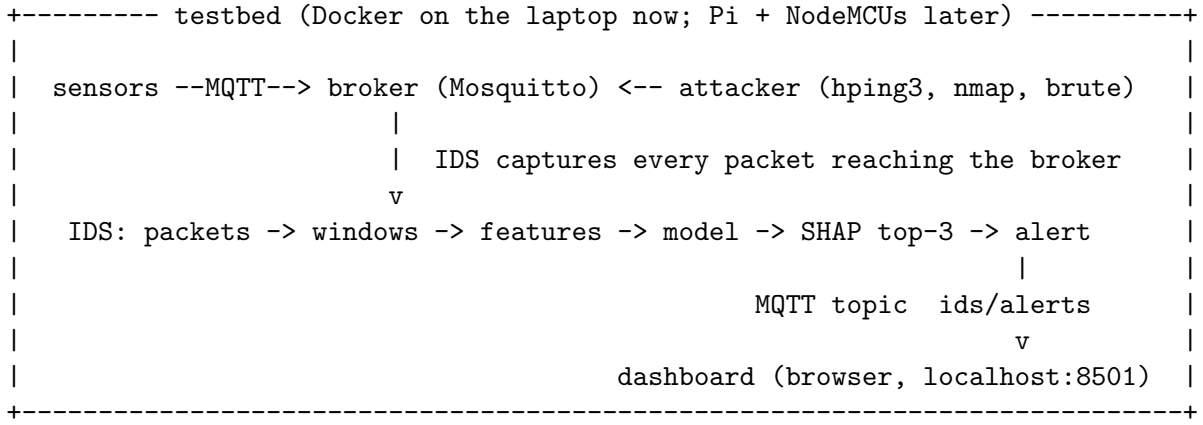
11 Build order **10**

12 Model choice for the first pipeline **10**

1 Why build the pipeline first

The model can be improved many times later. Once the pipeline exists, a better model is just a **file swap**, on one condition: the new model must use **the same feature list** the live feature extractor computes (section 8). So the order is: build the pipeline end-to-end with the run-2 models, then keep improving and compressing the models.

2 The big picture



MQTT = Message Queuing Telemetry Transport; **IDS** = Intrusion Detection System; **SHAP** = SHapley Additive exPlanations.

3 The five containers

Docker runs small isolated Linux "machines" (**containers**) on the laptop. They are joined by a private virtual network that exists only inside the laptop, so floods and scans are safe anywhere (never on the college network). All five start with one file, `docker-compose.yml`, and one command: `docker compose up`.

Container	Stands in for	What is inside	Job
broker	the Pi's Mosquitto	Official `eclipse-mosquitto` image; config with a username/password (needed for the brute-force demo)	Receives sensor data on port 1883
sensors	the 3 NodeMCUs	Python + `paho-mqtt`; 3 fake sensors publishing temperature / humidity every 2 s to `sensors/node1..3`	Normal (Benign) traffic
ids	the TinyIDS program on the Pi	Python + `scapy` (packet capture), LightGBM, ONNX Runtime, SHAP, `paho-mqtt`. **Shares the broker's network** (`network_mode: service:broker`), so it sees exactly what the Pi would see	Capture → features → model → SHAP → alert (section 4)
attacker	the laptop as attacker	Ubuntu + `hping3`, `nmap`, a Python MQTT brute-force script	Runs attacks on command, e.g. `docker compose exec attacker syn_flood`

Container	Stands in for	What is inside	Job
dashboard	the "Dashboard" box of the proposal	Python + **Streamlit** (turns a Python script into a web page) + `paho-mqtt`	Shows live alerts with their top-3 SHAP features (section 7)

4 Inside the IDS, step by step

1. **Capture**: `scapy` grabs every packet arriving at the broker (on the Pi: the `wlan0` interface).
2. **Group into windows per source IP**: packets from each sender are collected separately, so an alert can say *who* is attacking. Two window sizes run in parallel (section 5).
3. **Feature extractor**: for each full window, compute the features the model was trained on (`Rate`, `syn_count`, `Protocol Type`, `Tot size`, `Header_Length`, ...). **No** `IAT` (Inter-Arrival Time): run 2 showed it is a dataset shortcut.
4. **Model, in two steps**:
   - **Binary model**: "attack or normal?" (99.45% accurate in run 2; the reliable gate).
   - If attack: **family model**: "which kind?" (Flood / Mirai / Recon / Spoofing / Web / BruteForce).
5. **SHAP, only for windows flagged as attacks**: compute the top-3 features that caused the decision with **TreeSHAP**, and **time it**. That timing is RQ3 (Research Question 3, target < 50 ms).
6. **Alert**: publish to the MQTT topic `ids/alerts`, and keep running counters on `ids/stats` (packets/s, windows/s, model latency).
7. **Extra rule for MQTT brute force**: count failed MQTT logins (`CONNACK` refusals) per IP in a few seconds. This catches what the weak BruteForce class (F1 0.36 in run 2) misses.

Alert message format

```
{"time": "21:14:03", "src": "172.30.0.66", "window": 100,
 "attack": "Flood", "confidence": 0.98,
 "top3": [["syn_flag_number", 1.0, "+0.45"],
          ["Rate", 98000, "+0.31"],
          ["ack_count", 0, "+0.12"]],
 "model_ms": 1.4, "shap_ms": 12.0}
```

5 Packet windows: 10 and 100, like the dataset

The CICIoT2023 paper did not make one row per packet: it averaged the features over a **window of consecutive packets**, and used **two window sizes**:

Window	Used for these labels in the dataset
100 packets	all DDoS, DoS and Mirai attacks (floods produce huge numbers of packets)
10 packets	Benign, Recon, Spoofing, Web-based, BruteForce

So a "flood" row in the training data describes 100 packets, and a "port scan" row describes 10. The live IDS must produce rows that look like the training rows, so it uses **the same two windows**.

The problem

Live, we do not know in advance whether a sender is flooding or scanning, so we cannot pick one window size.

The solution: dual windows, per source IP

1. For every source IP keep **two counters at once**: a 10-packet window and a 100-packet window.
2. Every time a **10-packet window** fills: compute features, classify. This answers Benign / Recon / Spoofing / Web / BruteForce, and it reacts fast (a port scan is caught after 10 packets).
3. Every time a **100-packet window** fills: compute features, classify. This answers **Flood (DDoS/DoS) / Mirai**, exactly like the training rows.
4. **Decision rule**:
   - Flood / Mirai alerts are taken **only from 100-packet windows** (as in training).
   - Recon / Spoofing / Web / BruteForce / Benign are taken **only from 10-packet windows**.
   - A window whose prediction does not match its size is ignored (e.g. a 10-packet window that says "Flood").
5. **Slow senders**: a NodeMCU sends about 1 packet every 2 s, so a 100-packet window would take minutes. A **time-out** (e.g. 5 s) closes any half-full window so normal traffic is still checked regularly.

Floods are so fast (thousands of packets per second) that a 100-packet window fills in milliseconds, so detection is still immediate.

Matching the dataset's calculation

Per packet, compute the per-packet values (flags 0/1, protocol, size, header length, ...) and the per-flow counters (`syn_count`, `ack_count`, ..., kept per connection); then **average them over the window**, as the dataset authors did with pandas/NumPy. The paper does not publish formulas for `Rate` and `flow_duration`; we use the natural definitions (packets ÷ time, last time − first time). Live numbers will be close to, not identical with, the dataset (the proposal's "validity note").

6 Compression and quantization (RQ2): a required step

The proposal (Methodology step 4 and RQ2) requires shrinking the models before deployment and comparing them on the Pi:

> *"Prune tree depth and estimator count; convert the MLP/CNN to TensorFlow Lite or ONNX with INT8 quantization. On the Pi, measure model size (KB), peak RAM (MB), latency per flow (ms), throughput (flows/s) and energy per 1,000 inferences."*

Run 2's models are far too big (7–54 MB vs target < 1 MB), so this step is essential, not optional.

Where it sits in the pipeline

`train (20% / full data)` → `compress + quantize` → `export to models/current/` → `IDS loads it` → `benchmark on the Pi`

What is done to each model

Model	Compression	Result format
LightGBM (Light Gradient Boosting Machine)	**Pruning**: fewer trees (e.g. 50–150), fewer leaves (15–31), top-15 features from RQ1. **Quantization**: inputs are binned into at most 255 values (8 bits) and the thresholds/leaf values stored at lower precision.	LightGBM text model, and an **ONNX** file (Open Neural Network Exchange) for ONNX Runtime
Random Forest	**Pruning**: fewer trees (e.g. 20–50), max depth 10–14, top-15 features.	`.joblib` and ONNX
MLP (Multi-Layer Perceptron)	Export PyTorch → ONNX, then **INT8 quantization** (8-bit integers instead of 32-bit decimals) with ONNX Runtime. Optionally **pruning** of the weakest connections.	INT8 ONNX (≈ 4× smaller)
1D-CNN (one-dimensional Convolutional Neural Network)	Same as MLP: ONNX + INT8.	INT8 ONNX

What is measured (before and after compression)

- **Accuracy / macro-F1** on `test.csv`: how much is lost by compressing (should be small).
- **Model size (KB)**: target < 1 MB.
- On the Pi: **peak RAM (MB)** (target < 100 MB), **latency per window (ms)** (target < 5 ms), **throughput (windows/s)** (target ≥ 500), **energy per 1,000 inferences** with the USB power meter.

The result is the **RQ2 table**: which compressed model gives the best accuracy per millisecond and per megabyte on the Pi.

Two models for the IDS, one benchmark for RQ2

- The **IDS** uses the best compressed tree models (binary + family), because TreeSHAP explains trees quickly.

- **RQ2** compares all compressed models (tree ensemble, pruned MLP, small 1D-CNN) side by side, using a separate benchmark script on the Pi.

7 Swapping models: the contract

The IDS loads whatever is in `models/current/`:

File	Content
`binary.onnx / binary.txt`	Attack-or-normal model
`family.onnx / family.txt`	Which-attack model
`meta.json`	**Feature list in order**, class names, window sizes, model version, training date and scores

To use a better model: copy the new files into `models/current/`, restart the IDS container (`docker compose restart ids`). **Rule:** the features in `meta.json` must be features the extractor can compute. The training script will write `meta.json` automatically, and the IDS will refuse to start if a feature is unknown.

8 Where SHAP and the dashboard run

- **SHAP runs inside the IDS** (later on the Pi): it needs the model and the live features, and RQ3 asks for SHAP's speed *on the gateway*. It only runs for windows flagged as attacks, which keeps the Pi fast.
- **The dashboard only displays**. It subscribes to `ids/alerts`, `ids/stats` and `sensors/#` and shows:
  - a live **alert table** (time, attacker IP, attack type, confidence, window size);
  - for the selected alert, a **bar chart of its top-3 SHAP features** (the "reason");
  - a **traffic graph** (packets/s), so a flood is visible as a spike;
  - **model and SHAP latency** graphs (live RQ3 evidence);
  - the **sensor readings**, showing that normal traffic keeps flowing.
- Stage A: the dashboard is a container. Stage B: it runs **on the laptop** and listens to the Pi's broker, so the Pi does not spend power on drawing web pages.

9 Stage A (laptop only) and stage B (Pi + NodeMCUs)

	Stage A: laptop only (now, anywhere)	Stage B: lab or own hotspot
broker	container	**Pi** (Mosquitto already installed)
sensors	container (3 fake sensors)	**real NodeMCUs** (and the optional "master" botnet sketch)
IDS	container	**same Python code, run directly on the Pi**
attacker	container	**laptop** (WSL = Windows Subsystem for Linux, Ubuntu, with hping3 / nmap)
dashboard	container	**laptop**, listening to the Pi

	Stage A: laptop only (now, anywhere)	**Stage B: lab or own hotspot**
network	Docker private network inside the laptop	**own phone hotspot only**, never the college network

Windows limitation: Docker on Windows cannot properly see attacks coming from *outside* the laptop. Its port forwarding rebuilds connections, so a SYN flood from a real device would not arrive as a SYN flood. That is fine for stage A (everything inside Docker). For real NodeMCUs and real attacks, use stage B with the Pi, which is the actual project anyway.

10 Folder layout

```
project_final/testbed/
|-- docker-compose.yml          starts all five containers
|-- broker/     mosquitto.conf, passwords
|-- sensors/    sensor.py
|-- ids/        Dockerfile, ids.py, windows.py, features.py, explain.py
|-- attacker/   Dockerfile, syn_flood.sh, udp_flood.sh, port_scan.sh,
|               mqtt_bruteforce.py
|-- dashboard/  Dockerfile, app.py
|-- models/current/   binary + family model files, meta.json
`-- benchmark/  rq2_benchmark.py   (size / RAM / latency / energy, run on the Pi)
project_final/model/
|-- train_tinyids.py            training (already done: runs 1 and 2)
`-- compress.py                 pruning + ONNX export + INT8 quantization (new)
```

11 Build order

#	Step	Done when
1	broker + sensors + dashboard	sensor readings visible in the browser
2	IDS with the run-2 models, dual windows, no SHAP yet	live windows classified as "Benign"
3	attacker container: SYN flood, UDP flood, port scan	alerts appear (Flood, Flood, Recon)
4	SHAP top-3 in alerts + dashboard bar chart	every alert shows its reason; SHAP time logged
5	MQTT brute force + failed-login rule	brute force is alerted
6	`compress.py`: pruned trees, ONNX, INT8 MLP / CNN	models < 1 MB, F1 loss measured
7	swap compressed models into `models/current/`	IDS still detects all demo attacks
8	stage B: IDS on the Pi, NodeMCUs, laptop as attacker on a hotspot	live demo on real hardware
9	`rq2_benchmark.py` on the Pi + USB power meter	RQ2 table and RQ3 numbers
10	final full-data training of the chosen setup, swap in	final numbers for the report

12 Model choice for the first pipeline

- **Binary model**: LightGBM from run 2 (45 features, no `IAT`).
- **Family model**: **7 classes**, with DDoS and DoS merged into **"Flood"** (run 2 showed they cannot be separated without `IAT`), retrained quickly on the 20% sample.
- Both are replaced later by their compressed versions (step 6–7) and by the final full-data models (step 10).